MaskOff - Deepfake Detection System

Prepared By:
Harshil Vadalia (12202130501019)
Apoorva Wadaskar (12202130501003)

Objective:

To develop a web-based application that accurately detects fake images using a deep learning model trained on ResNet50. The goal is to provide a fast and reliable tool that can classify images as *REAL* or *FAKE* with high confidence, helping users identify manipulated or generated content through an intuitive interface.

Dataset Used:

We've used a dataset from Kaggle and then we pre processed it during training and then we trained our model. Here's a link of the dataset we used

NewData

Model Chosen:

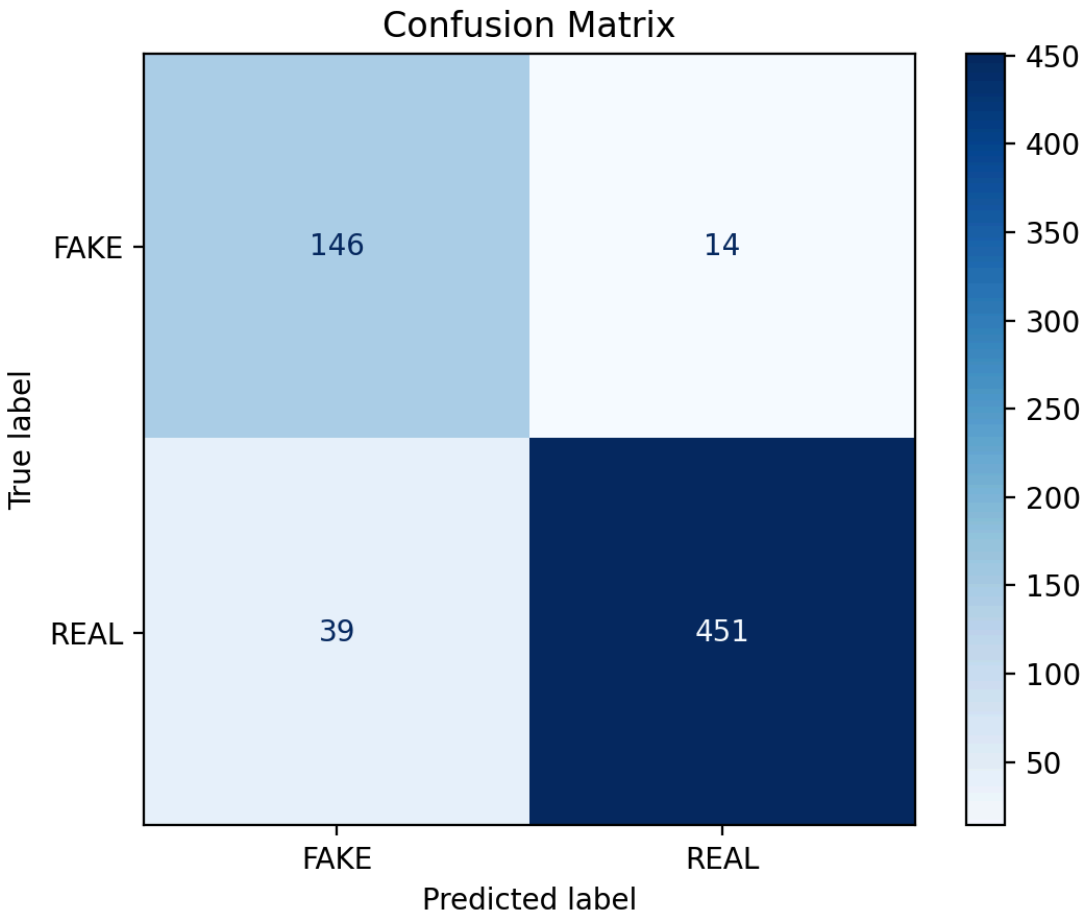
We've taken a pre-made ResNet-50 model and trained it with our dataset and we got 'resnet_model_new.h5' as our model.

Performance Metrics:

We've made a file called 'predict2.py' which evaluates model's F1 score and confusion matrix both. From this, we can determine how smoothly and efficiently our model is working.

F1 Score: 0.9445

Confusion Matrix:



Challenges & Learning:

1. **Model Training & Overfitting**
   Training ResNet50 on our dataset required careful tuning of parameters to avoid overfitting, especially given the limited size of real vs fake image datasets.

2. **Dataset Collection & Preprocessing**
   Ensuring the dataset had diverse and balanced examples of fake and real images was challenging. Preprocessing images into a consistent format without losing detail was also critical.

3. **Slow Inference Time**
   Initially, the model was loaded on every prediction request, which caused

significant delays during image processing in the web app.

4. **Flask & Frontend Integration**
   Integrating the backend model with a smooth frontend experience required managing file uploads, server-side routing, and client-side responsiveness carefully.

5. **Model File Size & Deployment**
   Handling a heavy `.h5` model file in a web app introduced issues related to memory and scalability, especially during local deployment or on limited-resource systems.

Learning:

1. **Transfer Learning with ResNet50**
   We learned how to leverage a pre-trained ResNet50 model and fine-tune it for a binary classification task involving fake vs real image detection.

2. **Efficient Model Deployment**
   Optimizing the prediction pipeline by loading the model once at startup significantly improved performance and taught us good practices for deploying ML models in web apps.

3. **Flask Web Development**
   Gained hands-on experience in building Flask APIs, handling HTTP requests, managing file uploads, and structuring backend code for production-readiness.

4. **Error Handling & Debugging**
   We encountered various runtime issues which helped us develop stronger debugging, logging, and exception-handling practices.

5. **Full-Stack Integration**
   Learned how to bridge the gap between machine learning and frontend development by integrating deep learning predictions into a real-time web interface.